



TECNOLÓGICO
NACIONAL DE MÉXICO



Tecnológico Nacional de México

Instituto Tecnológico de Saltillo

Ingeniería en Sistemas Computacionales

Cómputo en la Nube

Unidad 4

Práctica 4.2 – Inyección SQL

Alumna:

Silvia Guadalupe Ramos Mendoza 22050726

Docente: Ing. Miguel Salazar del Bosque

Saltillo, Coahuila; a 19 de marzo de 2026

Paso 1: Preparar el entorno

1.1 Configurar la base de datos

- Crear BD llamada vulnerable_db

```
silvia@srvbd:~$ sudo mariadb
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 31
Server version: 11.8.3-MariaDB-0+deb13u1 from Debian -- Please help get to 10k stars at https://github.com/MariaDB/Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE vulnerable_db;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> USE vulnerable_db;
```

- Crear una tabla llamada usuarios e insertar algunos registros de prueba

```
silvia@srvbd:~$ sudo mariadb
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 31
Server version: 11.8.3-MariaDB-0+deb13u1 from Debian -- Please help get to 10k stars at https://github.com/MariaDB/Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE vulnerable_db;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> USE vulnerable_db;
Database changed
MariaDB [vulnerable_db]> CREATE TABLE usuarios (
->   id INT AUTO_INCREMENT PRIMARY KEY,
->   nombre_usuario VARCHAR(50),
->   contrasena VARCHAR(50)
-> );
Query OK, 0 rows affected (0.051 sec)

MariaDB [vulnerable_db]> INSERT INTO usuarios (nombre_usuario, contrasena) VALUES ('admin', '1234');
Query OK, 1 row affected (0.013 sec)

MariaDB [vulnerable_db]> INSERT INTO usuarios (nombre_usuario, contrasena) VALUES ('user', 'password');
Query OK, 1 row affected (0.009 sec)

MariaDB [vulnerable_db]> |
```

```
silvia@srvbd:~$ mysql
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE vulnerable_db;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> USE vulnerable_db;
Database changed
MariaDB [vulnerable_db]> CREATE TABLE usuarios (
  ->   id INT AUTO_INCREMENT PRIMARY KEY,
  ->   nombre_usuario VARCHAR(50),
  ->   contrasena VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.051 sec)

MariaDB [vulnerable_db]> INSERT INTO usuarios (nombre_usuario, contrasena) VALUES ('admin', '1234');
Query OK, 1 row affected (0.013 sec)

MariaDB [vulnerable_db]> INSERT INTO usuarios (nombre_usuario, contrasena) VALUES ('user', 'password');
Query OK, 1 row affected (0.009 sec)

MariaDB [vulnerable_db]> SELECT * FROM usuarios;
+----+-----+-----+
| id | nombre_usuario | contrasena |
+----+-----+-----+
| 1  | admin         | 1234       |
| 2  | user          | password   |
+----+-----+-----+
2 rows in set (0.002 sec)

MariaDB [vulnerable_db]> |
```

```
silvia@srvbd:~$ sudo mariadb
The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Mar 19 07:08:14 2026 from 192.168.56.1
silvia@srvbd:~$ sudo mariadb
[sudo] contraseña para silvia:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 65
Server version: 11.8.3-MariaDB-0+deb13u1 from Debian -- Please help get to 10k stars at https://github.com/MariaDB/Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE USER 'vuln_user'@'192.168.56.112' IDENTIFIED BY 'Password123!';
Query OK, 0 rows affected (0.069 sec)
FLUSH PRIVILEGES;
EXIT;Query OK, 0 rows affected (0.011 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON vulnerable_db.* TO 'vuln_user'@'192.168.56.112';
Query OK, 0 rows affected (0.011 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.016 sec)

MariaDB [(none)]> EXIT;|
```

Paso 2: Crear la Aplicación Vulnerable

```
GNU nano 8.4 /var/www/html/login.php
<?php
$conn = new mysqli('192.168.56.111', 'vuln_user', 'Password123!', 'vulnerable_db');

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $usuario = $_POST['usuario'];
    $contrasena = $_POST['contrasena'];

    // Consulta vulnerable
    $sql = "SELECT * FROM usuarios WHERE nombre_usuario='$usuario' AND contrasena='$contrasena'";
    $result = $conn->query($sql);

    if ($result && $result->num_rows > 0) {
        echo "Login exitoso";
    } else {
        echo "Credenciales inválidas";
    }
}

<form method="POST" action="login.php">
    Usuario: <input type="text" name="usuario"><br>
    Contraseña: <input type="password" name="contrasena"><br>
</form>
```

Paso 3: Realizar la Inyección SQL

← → ↻ ⚠ No seguro 192.168.56.108/login.php

📏 | 📧 Gmail 📺 YouTube 📍 Maps 🍷 I wish, if only: deseo...

Usuario:

Contraseña:

Login exitoso

Usuario:

Contraseña:

Paso 4: Mitigación de la Inyección SQL

```
GNU nano 8.4 /var/www/html/login.php *
<?php
$conn = new mysqli('192.168.56.111', 'vuln_user', 'Password123!', 'vulnerable_db');

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $usuario = $_POST['usuario'];
    $contrasena = $_POST['contrasena'];
    // Consulta segura con prepared statements
    $stmt = $conn->prepare("SELECT * FROM usuarios WHERE nombre_usuario=?
AND contrasena=?");
    $stmt->bind_param("ss", $usuario, $contrasena);
    $stmt->execute();
    $result = $stmt->get_result();
    if ($result->num_rows > 0) {
        echo "Login exitoso";
    } else {
        echo "Credenciales inválidas";
    }
}
?>

<form method="POST" action="login.php">
    Usuario: <input type="text" name="usuario"><br>
```

- Repetir Inyección SQL

Usuario:

Contraseña:

Credenciales inválidas

Usuario:

Contraseña:

Conclusiones:

Esta práctica permitió entender de forma muy sencilla cómo es que funciona la inyección SQL. Al probar entradas como `admin' --`, se observa que es muy fácil romper la lógica del sistema y acceder sin contraseña; ya al implementar `bind_param`, el ataque dejó de funcionar, dejando como enseñanza que es muy importante la forma en la que se construyen las consultas para evitar este tipo de vulnerabilidades.

Es importante el desarrollo del código desde un inicio, tomando en cuenta la seguridad. Pequeños descuidos pueden generar riesgos muy grandes.